

EECS3311 Fall 2026
Course Project – Stage 1

Fashion Trend Intelligence + Microtrend Detection Agent

Student Name	Yeonsoo Lee
Student ID	220518676
Date	Oct 9 th , 2026

1. Project Overview

1.1 Problem and Motivation

What problem does the project solve?

Fashion trends move at a rapid pace, and it can be difficult to determine whether a style is emerging or already in the stage of declining. Trend signals can also be distributed across different sources, categories, and time periods, making manual analysis inconsistent and time-consuming.

The proposed system addresses this problem by analyzing historical fashion trend signals, detecting emerging microtrends, measuring trend momentum, classifying trend lifecycle stages, and generating evidence-based trend intelligence reports.

The motivation is to give users structured, evidence-based fashion trend AI intelligence rather than relying only on subjective opinions or sole social-media observations.

1.2 Target Users

Who are the target users?

The primary target users are:

- Fashion enthusiasts interested in emerging styles and aesthetics.
- Fashion content creators researching current and upcoming trends.

1.3 Agent Description

What can the agent do?

This agent can interpret natural-language fashion questions and coordinate retrieval, analysis, comparison, and AI interpretation. For example, if a user asks “Are low rise jeans still on trend?”, the agent identifies the requested trend, retrieves historical signals, calculates trend growths and the lifecycle, optionally compares related trends, and generates an evidence-based explanation.

Why is an AI agent appropriate for this problem?

Different fashion trend questions require different combinations of research steps. Instead of following one fixed process, the AI agent interprets the user’s intent, creates an appropriate plan, selects the required tools, executes multiple steps, and combines the results.

Step	Agent Action
1	Interpret the user's request submitted in natural language.
2	Create an analysis plan and select the required tools.
3	Retrieve historical fashion trend signals.
4	Normalize and validate the retrieved data.
5	Calculate trend momentum and growth metrics.
6	Classify the trend lifecycle.
7	Pass structured evidence to the LLM.
8	Generate and return the explanation or report.

1.4 AI/LLM Model(s)

Which AI/LLM model(s) do you plan to use?

The system will use OpenAI's GPT-6 Luna model (gpt-6-luna) through the OpenAI API. The model will interpret natural-language

fashion queries, support agent planning and tool selection, and generate evidence-based trend explanations. The application will interact with the model through the LLMClient interface, implemented by OpenAILLMClient.

1.5 Overall Architecture

How will the AI model interact with the rest of the software system?

The GUI and CLI submit commands to AgentController; each command delegates to the FashionTrendAgent facade. AgentPlanner uses LLMClient when interpreting a natural-language query and selects analysis steps. ToolManager then invokes the TrendRepository and TrendAnalyzer by tool name. TrendRepository loads locally imported, normalized CSV records; TrendAnalyzer calculates trend metrics and applies interchangeable ranking strategies. Only after evidence is produced does LLMClient generate a grounded explanation or report. The numerical values remain deterministic, and both interfaces receive the same AgentResponse. The initial implementation imports a locally stored fashion trend CSV dataset through an adapter.

Layer / Component	Responsibility
GUI / CLI	Display results and construct reusable commands from user requests.
AgentController	Executes a Command regardless of its originating interface.
FashionTrendAgent	Facade that coordinates plans, tool calls and AI explanations.
AgentPlanner / ToolManager	Plan request-specific tool steps; execute them using the repository and analyzer.
TrendRepository	Searches and stores trends and historical observations imported locally.
TrendDataImporter / CsvTrendAdapter	Converts CSV rows from CsvReader into validated TrendSignal records.
TrendAnalyzer	Calculates momentum, classifies lifecycle, finds microtrends and compares trends.
TrendRankingStrategy	Provides interchangeable momentum and growth-based ranking methods.
LLMClient / OpenAILLMClient	Interpret natural-language requests and summarize structured evidence.

2. Detailed Feature Specifications

F01 - Natural-Language Trend Query

Description: Allows users to ask fashion-related questions such as “What footwear trends are emerging?” The agent interprets the request and determines which analysis operations are needed.

User Interaction: The user enters a question in the GUI query box or submits it through the CLI.

Input: Natural-language fashion question.

Output: Structured trend analysis and AI-generated explanation.

AI Involvement: Hybrid

Workflow: The agent identifies the user’s intent and relevant trend/category, selects the required tools, executes retrieval and analysis, and sends the structured results to the LLM for interpretation.

Error/Alternative Cases: If the request is unclear or the trend cannot be found, the system asks for clarification or reports that insufficient data is available.

F02 - Trend Search and Retrieval

Description: Searches for a fashion trend and retrieves its available historical observations.

User Interaction: The user searches for a trend through the GUI search field or CLI.

Input: Trend name, optional category, and optional time range.

Output: Historical Trend Signal records.

AI Involvement: Deterministic.

Workflow: The repository or configured provider is queried, matching observations are retrieved, validated, and returned.

Error/Alternative Cases: If no trend or historical data is found, the system displays an informative message.

F03 - Trend Data Import and Normalization

Description: Imports fashion trend observations from a user-provided local CSV file and converts them into validated TrendSignal records.

User Interaction: The user selects Import CSV in the GUI or uses the CLI import command.

Input: CSV file path containing trend, category, date, popularity and source columns.

Output: Stored normalized trend signals and an import summary with skipped-row warnings.

AI Involvement: Deterministic.

Workflow: TrendRepository invokes a TrendDataImporter. CsvTrendAdapter reads rows using CsvReader, validates required fields, normalizes values, removes duplicates, and stores valid observations.

Error/Alternative Cases: Invalid rows are skipped with details; an unreadable file or zero valid rows produces a clear import error.

F04 - Trend Momentum Analysis

Description: Measures how strongly a fashion trend is growing or declining.

User Interaction: The user selects a trend to analyze through the GUI or CLI.

Input: Historical trend observations.

Output: Momentum score, short-term growth, long-term growth, and acceleration.

AI Involvement: Deterministic

Workflow: Historical values are sorted by date, growth and acceleration are calculated, and the results are combined into a normalized momentum score.

Error/Alternative Cases: If there is insufficient historical data, the system reports that momentum cannot be reliably calculated.

F05 - Trend Lifecycle Classification

Description: Classifies a fashion trend as Emerging, Growing, Peaking, or Declining.

User Interaction: The lifecycle status appears when the user analyzes or views a trend.

Input: Momentum, growth, acceleration, and historical popularity.

Output: Lifecycle stage and optional AI explanation.

AI Involvement: Hybrid

Workflow: Deterministic rules assign the lifecycle stage, and the LLM may explain the classification using the calculated metrics.

Error/Alternative Cases: If the available data is insufficient, the lifecycle is shown as unknown or low-confidence.

F06 - Emerging Microtrend Detection

Description: Identifies smaller trends showing strong recent growth before reaching high mainstream popularity.

User Interaction: The user selects an 'Emerging Trends' view or requests emerging trends for a category.

Input: Fashion category, candidate trends, and historical observations.

Output: Ranked list of emerging microtrends with momentum scores.

AI Involvement: Hybrid

Workflow: Candidate trends are retrieved, momentum is calculated for each, qualifying microtrends are ranked, and the LLM summarizes the strongest results.

Error/Alternative Cases: Trends with insufficient history are excluded; if too few candidates remain, the user is informed.

F07 - Category-Based Trend Discovery

Description: Lets users explore trends within categories such as footwear, colors, accessories, aesthetics, tops, and bottoms.

User Interaction: The user selects a category in the GUI or enters it through the CLI.

Input: Fashion category.

Output: Ranked list of trends with relevant metrics.

AI Involvement: Hybrid

Workflow: The system retrieves trends in the selected category, analyzes their momentum, ranks them, and optionally generates an AI summary.

Error/Alternative Cases: If the category is unsupported or contains no trend data, the system displays an appropriate message.

F08 - Trend Comparison

Description: Compares two fashion trends using the same analytical metrics.

User Interaction: The user selects two trends on the comparison screen or provides two trend names through the CLI.

Input: Two trend names and optional time range.

Output: Side-by-side comparison of momentum, growth, lifecycle, and historical performance.

AI Involvement: Hybrid.

Workflow: Both trends are retrieved and analyzed using the same logic, their results are compared, and the LLM summarizes the key differences.

Error/Alternative Cases: If one trend lacks enough data, the system provides a partial comparison or reports the limitation.

F09 - Trend History and Visualization

Description: Displays how a fashion trend's popularity has changed over time.

User Interaction: The user opens a trend history view in the GUI or requests historical values through the CLI.

Input: Trend name and optional date range.

Output: Time-series chart in the GUI and structured historical values in the CLI.

AI Involvement: Deterministic.

Workflow: Historical observations are retrieved, sorted chronologically, converted into time-series data, and displayed.

Error/Alternative Cases: Missing periods are shown as unavailable; The chart doesn't get generated if no history exists.

F10 - AI Trend Intelligence Report

Description: Generates a complete evidence-based report by coordinating several retrieval and analysis tools.

User Interaction: The user selects 'Generate Report' in the GUI or runs a report command in the CLI.

Input: Trend name and optional category or time range.

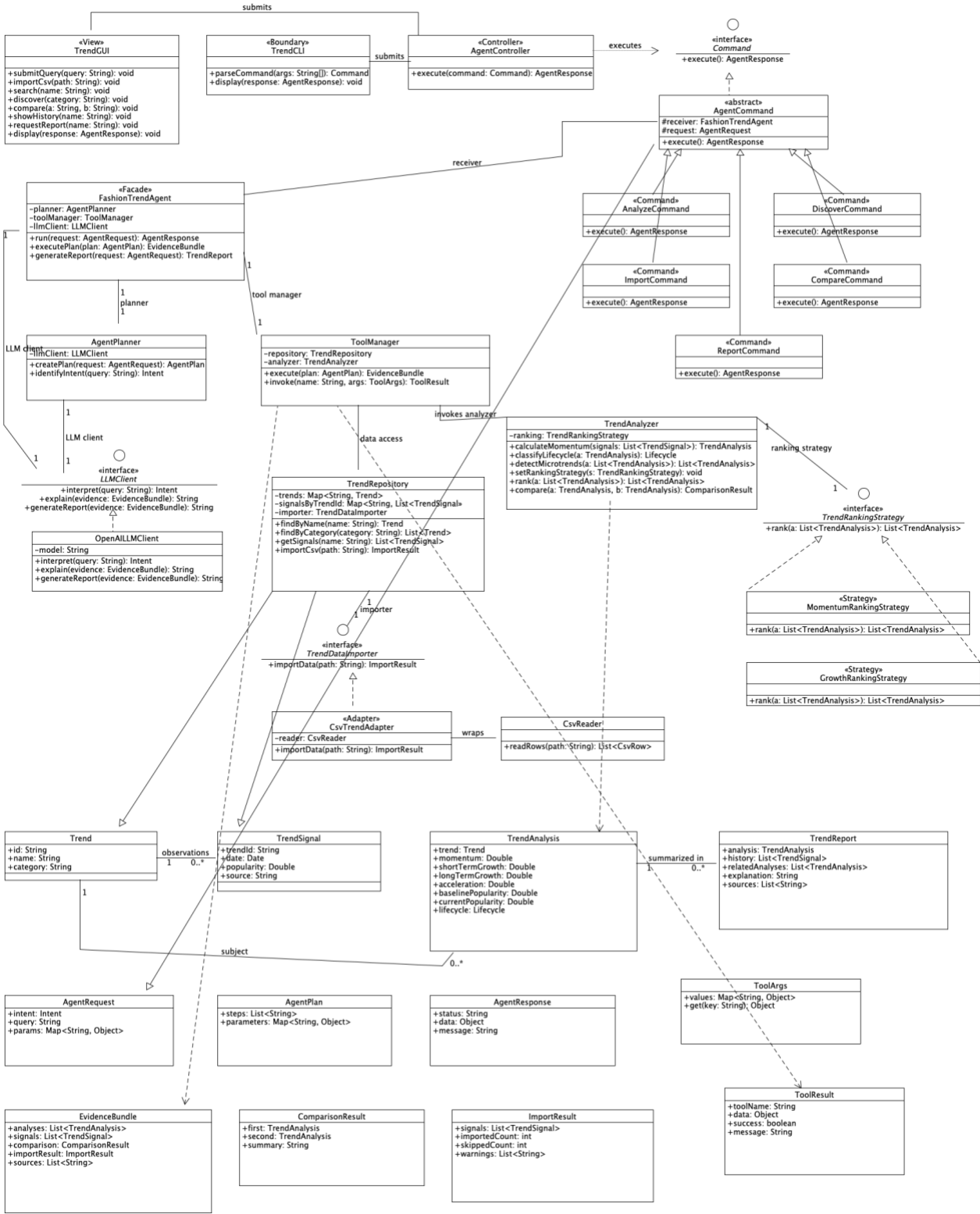
Output: Report containing momentum, lifecycle stage, growth, historical pattern, related trends, sources, and AI interpretation.

AI Involvement: Hybrid

Workflow: The agent plans the required steps, retrieves and aggregates trend data, runs momentum and lifecycle analysis, optionally compares related trends, and sends the combined results to the LLM for synthesis.

Error/Alternative Cases: If a data source or the LLM is unavailable, the system returns the available deterministic results and identifies the missing information.

3. UML Class Diagram



4. Design Pattern Explanations

4.1 MVC Pattern

Participating classes: TrendGUI (view); AgentController (controller); Trend, TrendSignal, TrendAnalysis, TrendReport and TrendRepository (model).

Problem addressed: Presentation and data-processing concerns must remain independent.

Roles: The view collects requests and renders charts; the controller executes the submitted command; the model holds trend data and analysis results.

Rationale: UI changes do not require changes to scoring or persistence.

Without this pattern: GUI event handlers would be tightly coupled to storage and calculation logic.

4.2 Command Pattern

Participating classes: Command; AgentCommand; AnalyzeCommand; DiscoverCommand; ImportCommand; CompareCommand; ReportCommand; AgentController; FashionTrendAgent.

Problem addressed: GUI and CLI need the same operations without duplicating control flow.

Roles: Command declares execute(); abstract AgentCommand holds an AgentRequest and the FashionTrendAgent receiver. Concrete commands invoke the appropriate facade operation. AgentController is the invoker.

Rationale: Inputs from both interfaces become interchangeable command objects that can be tested independently.

Without this pattern: Each interface would contain duplicated branching for analysis, import, discovery, comparison and reporting.

4.3 Facade Pattern

Participating classes: FashionTrendAgent; AgentPlanner; ToolManager; LLMClient; TrendRepository; TrendAnalyzer.

Problem addressed: A trend request may require planning, data retrieval, calculations and model interpretation.

Roles: FashionTrendAgent exposes run() and generateReport(), hiding the ordering and cooperation of lower-level components.

Rationale: Commands communicate with one high-level receiver rather than internal services.

Without this pattern: Controllers and UI code would need to know the internal tool workflow and handle its failure paths.

4.4 Adapter Pattern

Participating classes: TrendDataImporter (target); CsvTrendAdapter (adapter); CsvReader (adaptee); TrendRepository (client).

Problem addressed: Downloaded CSV rows do not match the application's TrendSignal data model.

Roles: CsvReader returns raw rows; CsvTrendAdapter converts, validates and deduplicates them through TrendDataImporter.importData(). TrendRepository depends on the interface.

Rationale: Source-specific import logic stays out of repository, analyzer and agent code.

Without this pattern: A CSV schema change would require changes throughout the analysis and persistence pipeline.

4.5 Strategy Pattern

Participating classes: TrendRankingStrategy; MomentumRankingStrategy; GrowthRankingStrategy; TrendAnalyzer.

Problem addressed: A category or discovery list can be ranked by different but comparable criteria.

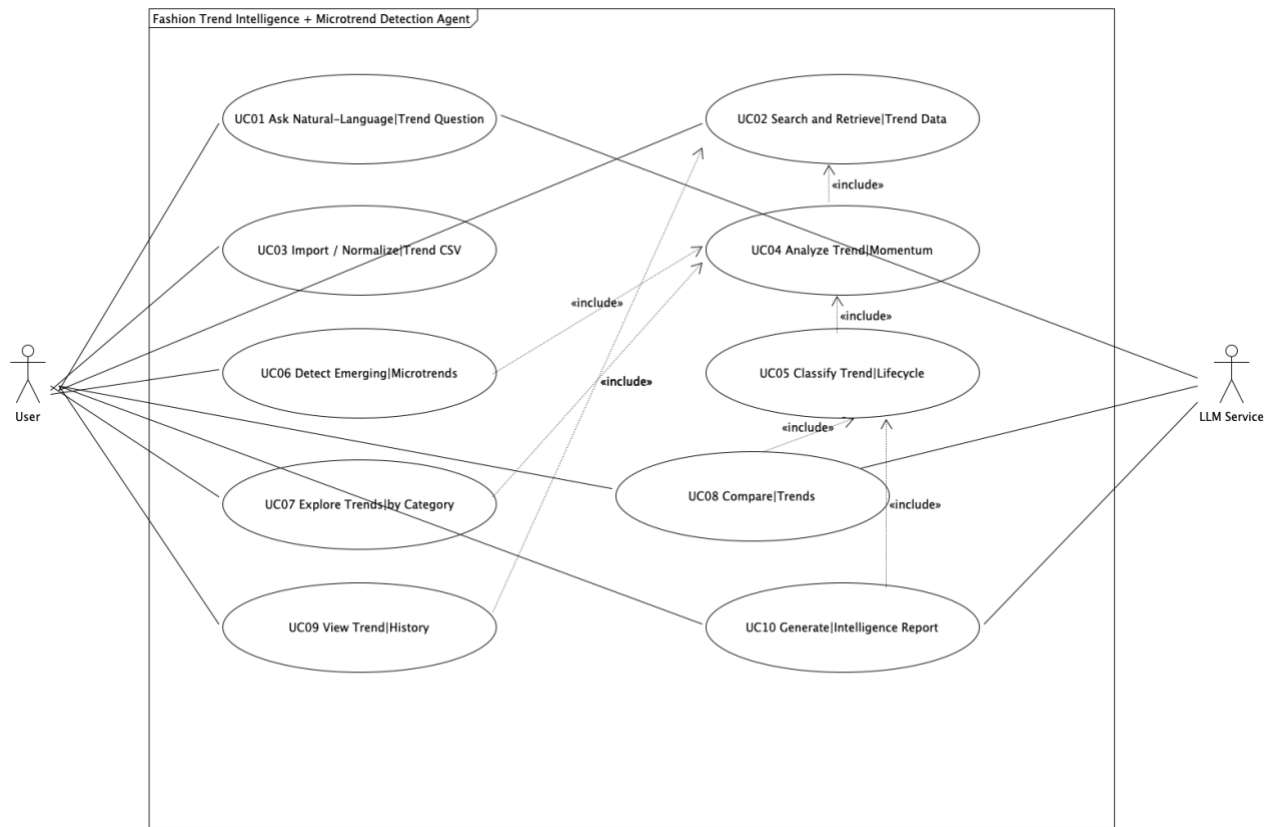
Roles: Both strategies implement rank(List<TrendAnalysis>); TrendAnalyzer holds a TrendRankingStrategy and can switch between them.

Rationale: Users can choose momentum- or growth-based ranking without changing TrendAnalyzer or its clients.

Without this pattern: Ranking selection would require conditionals throughout the analyzer. Momentum calculation and lifecycle classification remain separate sequential computations, not competing strategies.

5. Use-Case Diagram

The use-case diagram covers UC01–UC10, having actors as User and LLM Service. Required dependencies are shown with <<include>> where appropriate.



6. Detailed Use-Case Descriptions

UC01 - Ask Natural-Language Trend Question

Actor(s): User; LLM Service

Goal: Interpret a fashion question and return an evidence-grounded result.

Preconditions: App running; dataset imported; LLM configured for narrative queries.

Trigger: User submits a question.

Main Success Scenario:

1. TrendGUI or TrendCLI creates AnalyzeCommand with the question.
2. AgentController executes the command and FashionTrendAgent.run().
3. AgentPlanner asks LLMClient to interpret intent and creates an AgentPlan.
4. ToolManager invokes the repository and analyzer steps selected in the plan.
5. LLMClient explains the structured evidence and a common AgentResponse is displayed.

Alternative / Exception Flows:

- Unclear intent: ask for a specific trend or category.

- Missing trend or history: show insufficient data.
- Unavailable LLM: show deterministic results when a query can still be resolved.

Postconditions: User receives a structured response.

Related Feature(s): F01

UC02 - Search and Retrieve Trend Data

Actor(s): User

Goal: Find a fashion trend and its stored historical observations.

Preconditions: At least one successful data import.

Trigger: User searches, or another operation requests data.

Main Success Scenario:

1. Search request reaches FashionTrendAgent through AnalyzeCommand.
2. ToolManager calls TrendRepository.findByName().
3. TrendRepository.getSignals() returns matching records.
4. Result is returned to GUI or CLI.

Alternative / Exception Flows:

- Unknown trend: offer no-match notice.
- No history in selected range: return empty state.

Postconditions: Stored observations are available for analysis.

Related Feature(s): F02

UC03 — Import and Normalize Trend CSV

Actor(s): User

Goal: Load clean historical trend data for subsequent analysis.

Preconditions: Readable CSV file available locally.

Trigger: User selects Import CSV or invokes the CLI import command.

Main Success Scenario:

1. GUI/CLI creates ImportCommand.
2. ToolManager asks TrendRepository.importCsv(path).
3. TrendRepository calls TrendDataImporter.importData(path).
4. CsvTrendAdapter uses CsvReader to parse raw rows and normalize data.
5. TrendRepository stores valid records and returns counts and warnings.

Alternative / Exception Flows:

- Unreadable file: import fails with useful error.
- Malformed rows: skip and report warnings.
- No valid observations: reject import.

Postconditions: Valid normalized TrendSignal records are available in the repository.

Related Feature(s): F03

UC04 — Analyze Trend Momentum

Actor(s): User

Goal: Measure recent movement of a trend.

Preconditions: Enough chronological observations to calculate rates.

Trigger: User requests momentum or another use case needs it.

Main Success Scenario:

1. ToolManager obtains the historical signals from TrendRepository.
2. TrendAnalyzer.calculateMomentum() computes recent and longer-term growth and acceleration.
3. TrendAnalysis returns the computed metrics.

Alternative / Exception Flows:

- Insufficient observations: metrics unavailable.
- Bad or missing values: exclude if enough valid points remain.

Postconditions: TrendAnalysis holds growth and momentum values.

Related Feature(s): F04

UC05 — Classify Trend Lifecycle

Actor(s): User; LLM Service (optional)

Goal: Identify Emerging, Growing, Peaking or Declining status.

Preconditions: TrendAnalysis contains usable momentum and history.

Trigger: User views classification or another workflow requests it.

Main Success Scenario:

1. UC04 computes the necessary trend metrics.
2. TrendAnalyzer.classifyLifecycle() applies consistent classification rules.
3. The classification is attached to TrendAnalysis.
4. LLMClient may explain the result from the metrics.

Alternative / Exception Flows:

- Unreliable classification: label as Unknown.
- LLM unavailable: show the deterministic classification alone.

Postconditions: Lifecycle stage is available.

Related Feature(s): F05

UC06 — Detect Emerging Microtrends

Actor(s): User; LLM Service (optional)

Goal: Identify fast-growing smaller trends within a category.

Preconditions: Category contains candidates with adequate history.

Trigger: User requests emerging microtrends.

Main Success Scenario:

1. DiscoverCommand delegates to FashionTrendAgent.
2. ToolManager retrieves category trends and computes each momentum score.

3. TrendAnalyzer.detectMicrotrends() filters high-growth/low-baseline candidates.
4. TrendAnalyzer.rank() orders candidates; optional LLM summary is produced.

Alternative / Exception Flows:

- Insufficient candidates: show warning.
- Unavailable LLM: return ranked metrics without prose.

Postconditions: Ranked microtrend list is displayed.

Related Feature(s): F06

UC07 — Explore Trends by Category

Actor(s): User; LLM Service (optional)

Goal: Rank and examine trends for a fashion category.

Preconditions: Category exists in dataset.

Trigger: User selects category in GUI/CLI.

Main Success Scenario:

1. DiscoverCommand delegates the category request.
2. ToolManager retrieves category trends and historical observations.
3. TrendAnalyzer calculates metrics for each trend.
4. Selected ranking strategy orders results; optional summary is displayed.

Alternative / Exception Flows:

- Unsupported category: show available categories.
- Data too sparse: omit unrankable candidates.

Postconditions: A category trend ranking is shown.

Related Feature(s): F07

UC08 — Compare Trends

Actor(s): User; LLM Service

Goal: Compare two trends on consistent metrics.

Preconditions: Both names resolve to known trends.

Trigger: User selects two trends.

Main Success Scenario:

1. CompareCommand calls FashionTrendAgent.run().
2. ToolManager loads both trend histories.
3. TrendAnalyzer computes momentum and lifecycle for both.
4. TrendAnalyzer.compare() constructs ComparisonResult.
5. LLMClient explains differences and the GUI/CLI presents a comparison.

Alternative / Exception Flows:

- One trend lacks history: explain incomplete comparison.
- Ambiguous name: request clarification.

Postconditions: Comparison result is available.

Related Feature(s): F08

UC09 — View Trend History

Actor(s): User

Goal: Show chronological historical popularity values.

Preconditions: Trend exists in repository.

Trigger: User opens the trend history screen / CLI command.

Main Success Scenario:

1. AnalyzeCommand sends a HISTORY request to FashionTrendAgent.
2. ToolManager invokes TrendRepository.getSignals().
3. Records are ordered by date and returned.
4. GUI draws a chart; CLI formats the same observations as a table.

Alternative / Exception Flows:

- Empty period: display no-data state.

Postconditions: Historical data is displayed without mutation.

Related Feature(s): F09

UC10 — Generate Trend Intelligence Report

Actor(s): User; LLM Service

Goal: Create an evidence-grounded report for a selected trend.

Preconditions: Trend exists; enough historical data; LLM available for narrative.

Trigger: User chooses Generate Report.

Main Success Scenario:

1. ReportCommand invokes FashionTrendAgent.generateReport().
2. AgentPlanner creates a retrieval/analysis plan.
3. ToolManager loads signals, runs momentum and lifecycle steps and optionally finds related trends.
4. LLMClient.generateReport() summarizes the structured EvidenceBundle.
5. TrendReport is returned and displayed.

Alternative / Exception Flows:

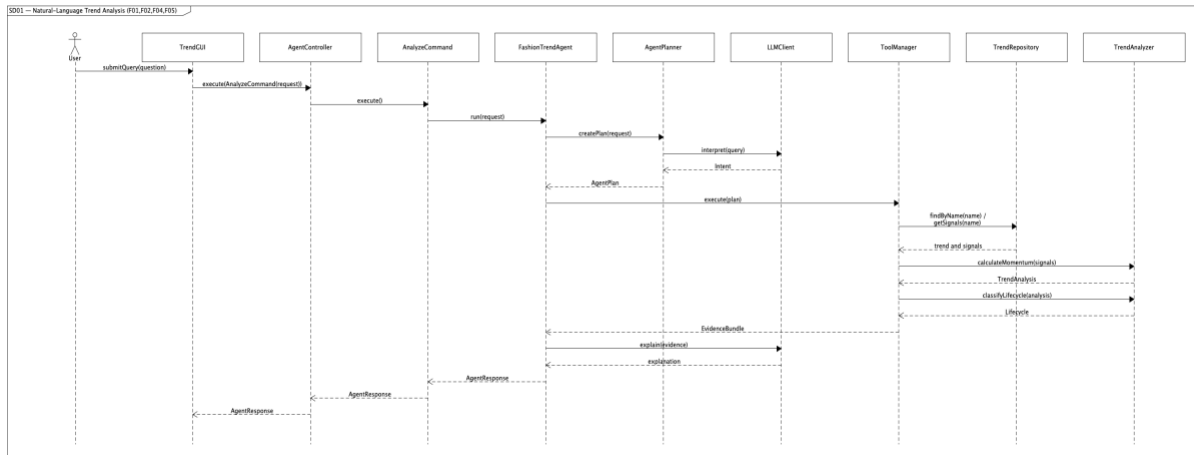
- Missing data: report unavailable metrics explicitly.
- LLM fails: return numeric report without AI narrative.

Postconditions: TrendReport is available for review.

Related Feature(s): F10

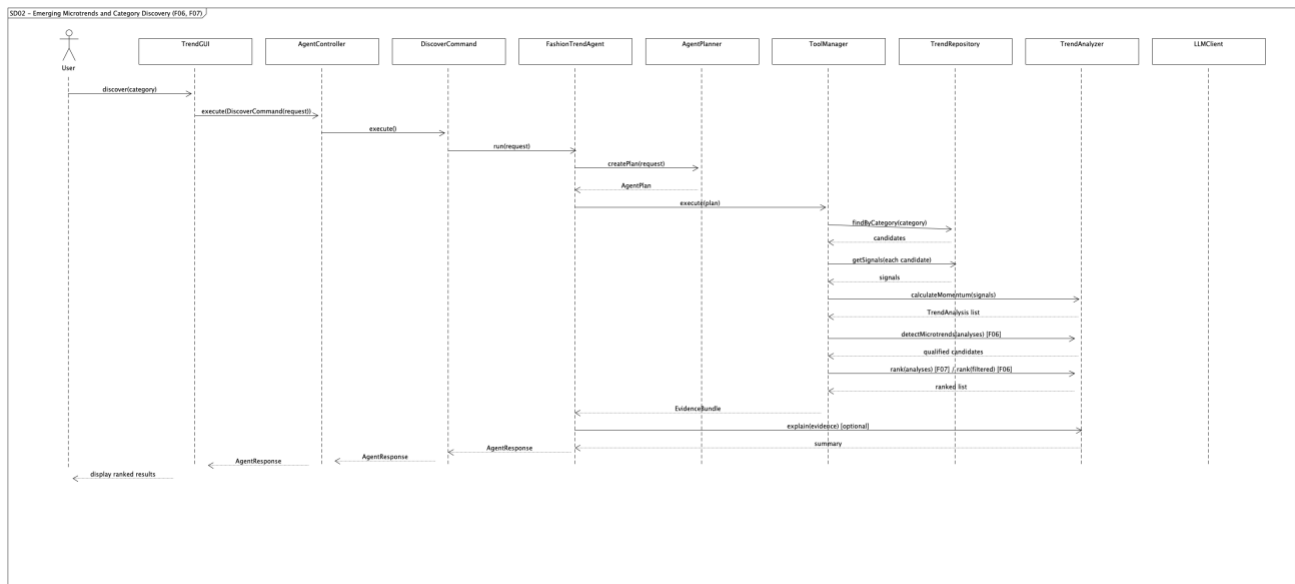
7. Sequence Diagrams

SD01 — Natural-Language Analysis



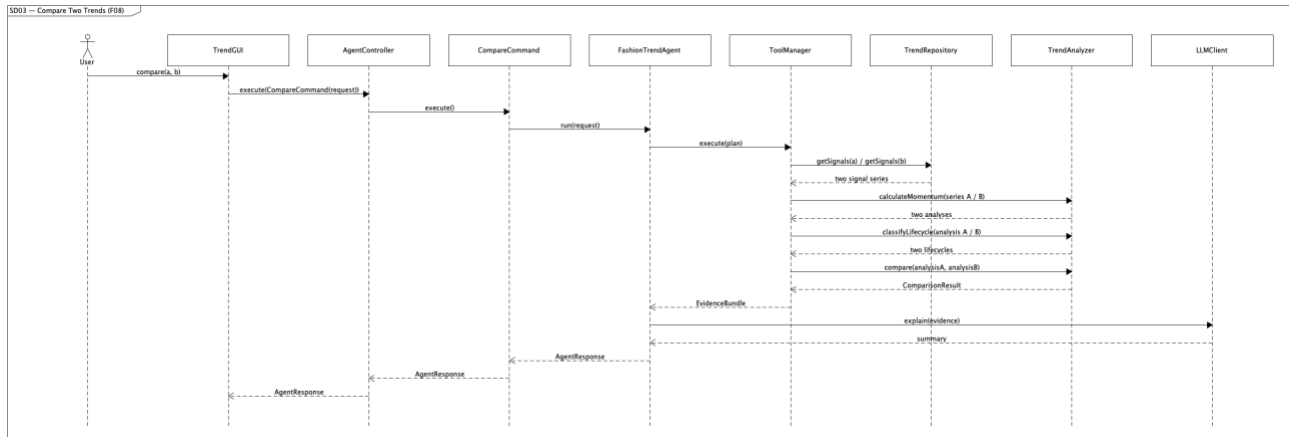
Covers F01, F02, F04, F05

SD02 — Microtrend / Category Discovery



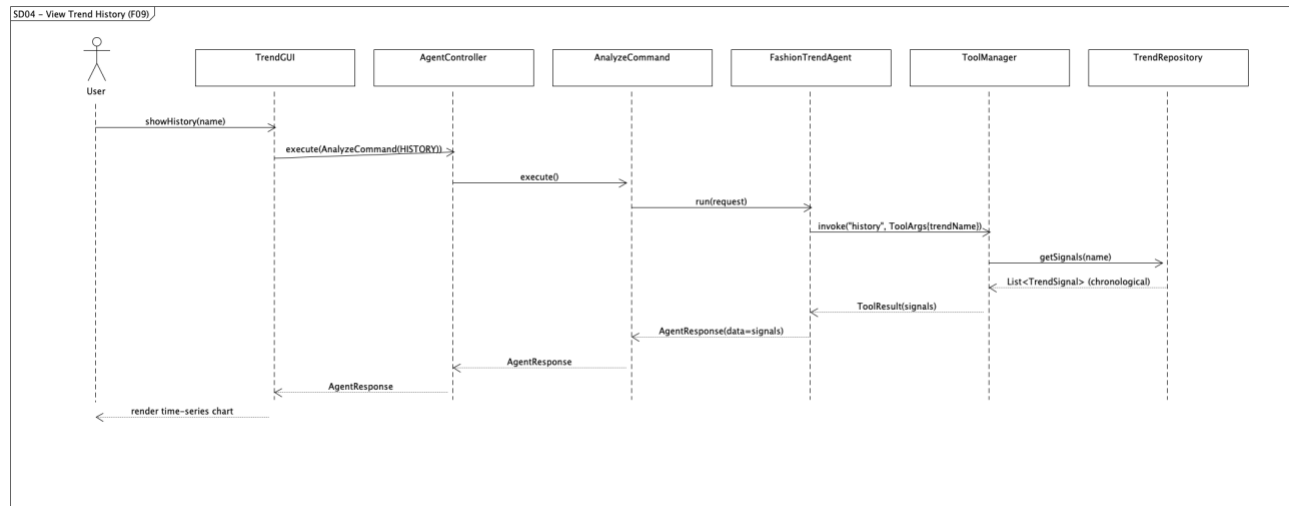
Covers F06, F07

SD03 — Trend Comparison



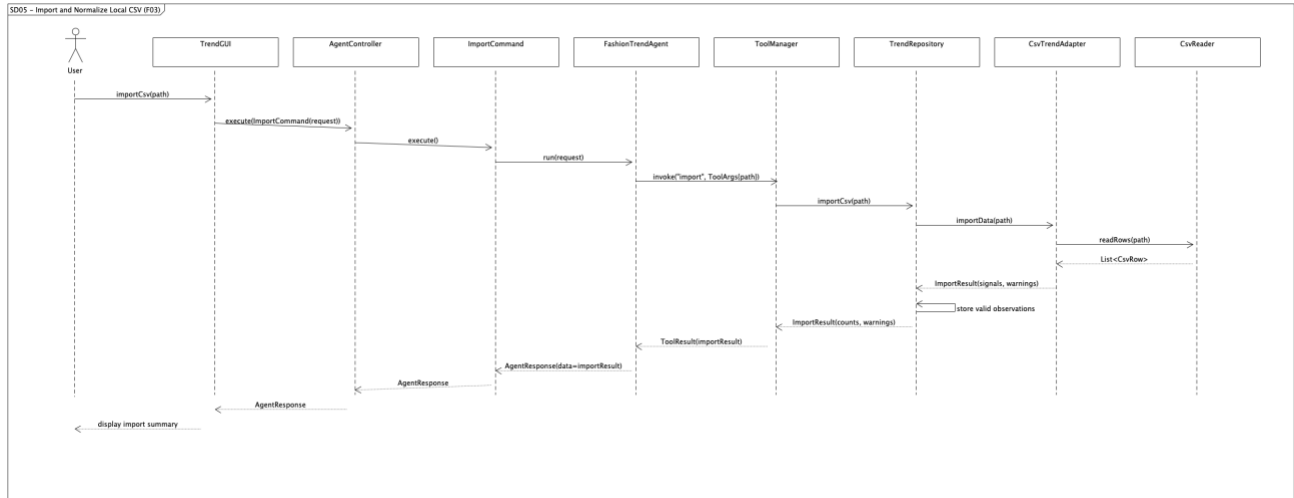
Covers F08

SD04 — Trend History



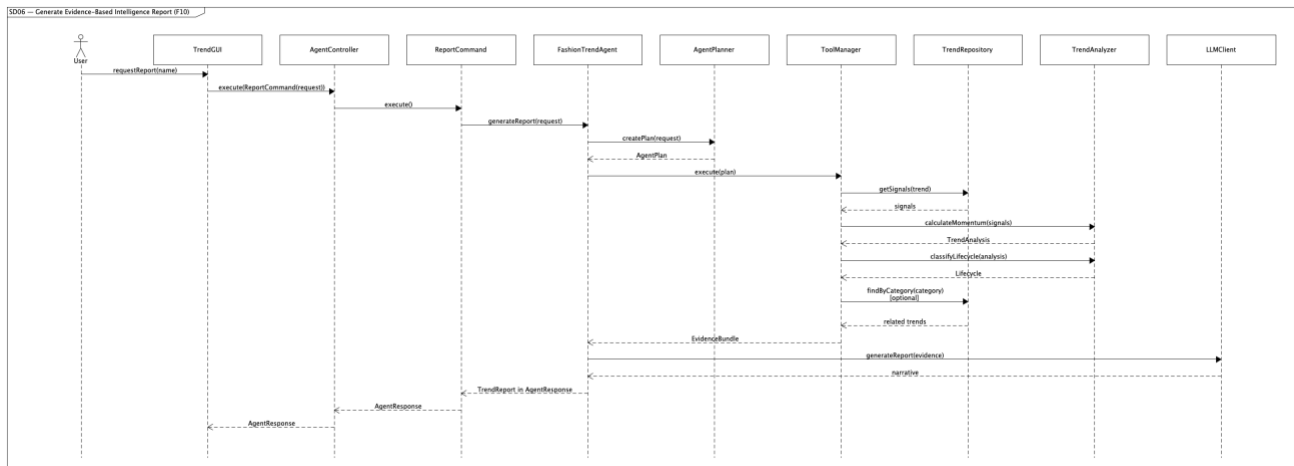
Covers F09

SD05 — Local CSV Data Import



Covers F03

SD06 — Trend Intelligence Report



Covers F10

8. Feature-to-Design Traceability Table

Feature	Description / Type	Use Case	Classes	Key Methods	Sequence	Pattern(s)
F01	Natural-Language Trend Query / Hybrid	UC01	TrendGUI; AgentController; AnalyzeCommand; FashionTrendAgent; AgentPlanner; LLMClient	submitQuery(); execute(); run(); createPlan(); interpret(); explain()	SD01	MVC; Command; Facade
F02	Trend Search and Retrieval / Deterministic	UC02	AnalyzeCommand; FashionTrendAgent; ToolManager; TrendRepository	execute(); run(); invoke(); findByName(); getSignals()	SD01	Command; Facade
F03	Trend Data Import and Normalization / Deterministic	UC03	ImportCommand; ToolManager; TrendRepository; TrendDataImporter; CsvTrendAdapter; CsvReader	execute(); invoke(); importCsv(); importData(); readRows()	SD05	Command; Adapter

F04	Trend Momentum Analysis / Deterministic	UC04	TrendRepository; ToolManager; TrendAnalyzer; TrendAnalysis	getSignals(); invoke(); calculateMomentum()	SD01	Facade
F05	Trend Lifecycle Classification / Hybrid	UC05	TrendAnalyzer; LLMClient; TrendAnalysis	classifyLifecycle(); explain()	SD01	Facade
F06	Emerging Microtrend Detection / Hybrid	UC06	DiscoverCommand; TrendRepository; TrendAnalyzer; TrendRankingStrategy; LLMClient	execute(); findByCategory(); calculateMomentum(); detectMicrotrends(); rank(); explain()	SD02	Command; Strategy
F07	Category-Based Trend Discovery / Hybrid	UC07	DiscoverCommand; ToolManager; TrendRepository; TrendAnalyzer; TrendRankingStrategy	execute(); invoke(); findByCategory(); rank()	SD02	Command; Strategy
F08	Trend Comparison / Hybrid	UC08	CompareCommand; FashionTrendAgent; TrendAnalyzer; LLMClient	execute(); run(); calculateMomentum(); classifyLifecycle(); compare(); explain()	SD03	Command; Facade
F09	Trend History and Visualization / Deterministic	UC09	TrendGUI; AnalyzeCommand; TrendRepository	showHistory(); execute(); getSignals()	SD04	MVC; Command
F10	AI Trend Intelligence Report / Hybrid	UC10	ReportCommand; FashionTrendAgent; AgentPlanner; ToolManager; TrendAnalyzer; LLMClient; TrendReport	execute(); generateReport(); createPlan(); execute(); calculateMomentum(); classifyLifecycle(); generateReport()	SD06	Command; Facade

9. Feature Implementation Explanations

F01 — Natural-Language Trend Query

Related Use Case: UC01

Related Sequence Diagram: SD01

Classes involved: TrendGUI, AnalyzeCommand, AgentController, FashionTrendAgent, AgentPlanner, ToolManager, LLMClient

Important methods: submitQuery(), execute(), run(), createPlan(), interpret(), execute(), explain()

Execution: The boundary creates an AnalyzeCommand from the user question. The controller invokes it, the facade creates an LLM-assisted plan and executes selected tools, and the LLM explains the resulting evidence.

F02 — Trend Search and Retrieval

Related Use Case: UC02

Related Sequence Diagram: SD01

Classes involved: AnalyzeCommand, FashionTrendAgent, ToolManager, TrendRepository

Important methods: execute(), run(), invoke(), findByName(), getSignals()

Execution: The request invokes repository search and retrieval through ToolManager. Matching historical TrendSignal records are returned to the requester.

F03 — Trend Data Import and Normalization

Related Use Case: UC03

Related Sequence Diagram: SD05

Classes involved: ImportCommand, FashionTrendAgent, ToolManager, TrendRepository, TrendDataImporter, CsvTrendAdapter, CsvReader

Important methods: execute(), run(), invoke(), importCsv(), importData(), readRows()

Execution: The import command asks ToolManager to import a CSV. TrendRepository depends on the TrendDataImporter interface; CsvTrendAdapter wraps CsvReader, normalizes records and returns valid data plus warnings.

F04 — Trend Momentum Analysis

Related Use Case: UC04

Related Sequence Diagram: SD01

Classes involved: ToolManager, TrendRepository, TrendAnalyzer, TrendAnalysis

Important methods: invoke(), getSignals(), calculateMomentum()

Execution: The repository supplies a chronological signal series. TrendAnalyzer computes short- and long-term growth, acceleration and momentum in a TrendAnalysis value.

F05 — Trend Lifecycle Classification

Related Use Case: UC05

Related Sequence Diagram: SD01

Classes involved: TrendAnalyzer, TrendAnalysis, LLMClient

Important methods: classifyLifecycle(), explain()

Execution: After momentum is computed, TrendAnalyzer applies lifecycle rules. The assigned stage may be explained by LLMClient without changing the numeric result.

F06 — Emerging Microtrend Detection

Related Use Case: UC06

Related Sequence Diagram: SD02

Classes involved: DiscoverCommand, TrendRepository, TrendAnalyzer, TrendRankingStrategy, LLMClient

Important methods: execute(), findByCategory(), calculateMomentum(), detectMicrotrends(), rank(), explain()

Execution: Category candidates are analyzed individually; TrendAnalyzer filters trends with unusually high growth and comparatively low popularity, ranks qualifying candidates with a ranking strategy and optionally requests a summary.

F07 — Category-Based Trend Discovery

Related Use Case: UC07

Related Sequence Diagram: SD02

Classes involved: DiscoverCommand, TrendRepository, TrendAnalyzer, TrendRankingStrategy

Important methods: execute(), findByCategory(), calculateMomentum(), rank()

Execution: The selected category supplies candidate trends. TrendAnalyzer computes metrics, then delegates ranking to MomentumRankingStrategy or GrowthRankingStrategy.

F08 — Trend Comparison

Related Use Case: UC08

Related Sequence Diagram: SD03

Classes involved: CompareCommand, FashionTrendAgent, TrendRepository, TrendAnalyzer, LLMClient

Important methods: execute(), run(), getSignals(), calculateMomentum(), classifyLifecycle(), compare(), explain()

Execution: The comparison command retrieves both trend histories; the same calculations are applied to each before TrendAnalyzer builds a comparison and LLMClient explains differences.

F09 — Trend History and Visualization

Related Use Case: UC09

Related Sequence Diagram: SD04

Classes involved: TrendGUI, AnalyzeCommand, FashionTrendAgent, TrendRepository

Important methods: showHistory(), execute(), run(), getSignals()

Execution: A HISTORY operation retrieves the chronological signals; the GUI plots them and the CLI prints the same data. No AI call is required.

F10 — AI Trend Intelligence Report

Related Use Case: UC10

Related Sequence Diagram: SD06

Classes involved: ReportCommand, FashionTrendAgent, AgentPlanner, ToolManager, TrendAnalyzer, LLMClient, TrendReport

Important methods: execute(), generateReport(), createPlan(), execute(), calculateMomentum(), classifyLifecycle(), generateReport()

Execution: The report command initiates a multi-step plan. ToolManager retrieves history and calculates structured metrics; LLMClient generates narrative text grounded in that evidence, and TrendReport stores the final output.